EXPERIMENT - 7

1. In a 3D modeling application, define an abstract class named Solid that declares an abstract method calculateVolume(). Then, derive two concrete classes—Cuboid and Sphere—from the Solid class. Each subclass should override calculateVolume() to compute its volume using the appropriate geometric formula. Finally, in your main method, create objects of Cuboid and Sphere and call their calculateVolume() methods to display the computed volumes.

```
abstract class Solid {
    abstract double calculateVolume();
}

class Cuboid extends Solid {
    private double length, width, height;

    public Cuboid(double length, double width, double height) {
        this.length = length;
        this.width = width;
        this.height = height;
    }

    @Override
    double calculateVolume() {
        return length * width * height;
    }
}

class Sphere extends Solid {
    private double radius;

    public Sphere(double radius) {
        this.radius = radius;
    }

    @Override
    double calculateVolume() {
        return (4.0 / 3.0) * Math.PI * Math.pow(radius, 3);
    }
```

```
}

public class Main {
    public static void main(String[] args) {
        Solid cuboid = new Cuboid(5.0, 3.0, 2.0);
        Solid sphere = new Sphere(4.0);

        System.out.printf("Cuboid Volume: %.2f\n", cuboid.calculateVolume());
        System.out.printf("Sphere Volume: %.2f\n", sphere.calculateVolume());
    }
}
```

```
Cuboid Volume: 30.00
Sphere Volume: 268.08

=== Code Execution Successful ===
```

2. Develop an abstract class called Worker with two abstract methods: computePay() and displayInfo(). Next, create two subclasses named FullTimeWorker and PartTimeWorker that extend Worker. Implement the computePay() method in each subclass—for example, by calculating a fixed monthly wage for full-time workers and an hourly rate for part-time workers—and implement displayInfo() to print out the worker's details. In your main method, instantiate objects of both subclasses and test their functionality to demonstrate polymorphism and code reuse

```
abstract class Worker {
    String name;
    int id;

    public Worker(String name, int id) {
        this.name = name;
```

```
        this.id = id;
    }

    abstract double computePay();
    abstract void displayInfo();
}

// Full-time worker subclass
class FullTimeWorker extends Worker {
    double monthlySalary;

    public FullTimeWorker(String name, int id, double monthlySalary) {
        super(name, id);
        this.monthlySalary = monthlySalary;
    }

    @Override
    double computePay() {
        return monthlySalary;
    }

    @Override
    void displayInfo() {
        System.out.println("Full-Time Worker:");
            System.out.println("Name: " + name + ", ID: " + id + ", Monthly Pay: $" +
computePay());
    }
}

// Part-time worker subclass
class PartTimeWorker extends Worker {
    double hourlyRate;
    int hoursWorked;

    public PartTimeWorker(String name, int id, double hourlyRate, int hoursWorked) {
        super(name, id);
        this.hourlyRate = hourlyRate;
        this.hoursWorked = hoursWorked;
    }

    @Override
    double computePay() {
        return hourlyRate * hoursWorked;
    }
```

```
    @Override
    void displayInfo() {
        System.out.println("Part-Time Worker:");
            System.out.println("Name: " + name + ", ID: " + id + ", Weekly Pay: $" +
computePay());
    }
}

public class Main {
    public static void main(String[] args) {
        Worker fullTime = new FullTimeWorker("Alice", 101, 4000.0);
        Worker partTime = new PartTimeWorker("Bob", 202, 20.0, 25);

        // Demonstrating polymorphism
        fullTime.displayInfo();
        partTime.displayInfo();
    }
}
```

```
Full-Time Worker:
Name: Alice, ID: 101, Monthly Pay: $4000.0
Part-Time Worker:
Name: Bob, ID: 202, Weekly Pay: $500.0

=== Code Execution Successful ===
```

3. Design an interface called Wallet that declares two methods: addFunds(double amount) and spendFunds(double amount). Implement this interface in a class named DigitalWallet that maintains a private balance variable. Ensure that the class provides controlled access to update the balance only through the interface methods. Then, write a separate class with a main method that creates a DigitalWallet object, performs a series of fund additions and expenditures, and prints the

updated balance to verify that external classes cannot directly manipulate the wallet's internal data.

```
// Interface declaring wallet operations
interface Wallet {
    void addFunds(double amount);
    void spendFunds(double amount);
}

// DigitalWallet class implementing Wallet interface
class DigitalWallet implements Wallet {
    private double balance;

    public DigitalWallet() {
        this.balance = 0.0;
    }

    @Override
    public void addFunds(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Added: $" + amount);
        } else {
            System.out.println("Amount must be positive.");
        }
    }

    @Override
    public void spendFunds(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            System.out.println("Spent: $" + amount);
        } else {
            System.out.println("Insufficient funds or invalid amount.");
        }
    }

    public double getBalance() {
        return balance;
    }
```

```
}

// Main class to test the DigitalWallet
public class Main {
    public static void main(String[] args) {
        DigitalWallet myWallet = new DigitalWallet();

        myWallet.addFunds(100.0);
        myWallet.spendFunds(30.0);
        myWallet.spendFunds(80.0); // Will fail due to insufficient funds
        myWallet.addFunds(50.0);

        System.out.println("Final Balance: $" + myWallet.getBalance());

        // The following line would produce an error if uncommented:
        // myWallet.balance = 1000; // Cannot access private field
    }
}
```

```
Added: $100.0
Spent: $30.0
Insufficient funds or invalid amount.
Added: $50.0
Final Balance: $120.0
```

4. Create an interface named Remote with methods powerOn(), powerOff(), and changeChannel(int channel). Next, implement this interface in a class called Television, where each method prints a message indicating its action (e.g., "TV is now on", "TV is off", "Channel changed to X"). Finally, develop a class with a main method that instantiates a Television object and uses a Remote reference to call the methods, thereby demonstrating how polymorphism and interface abstraction allow you to control a device through a common interface.

```
// Define the interface
interface Remote {
    void powerOn();
    void powerOff();
    void changeChannel(int channel);
}

// Implement the interface in the Television class
class Television implements Remote {
    private boolean isOn = false;
    private int currentChannel = 1;

    @Override
    public void powerOn() {
        isOn = true;
        System.out.println("TV is now ON.");
    }

    @Override
    public void powerOff() {
        isOn = false;
        System.out.println("TV is now OFF.");
    }

    @Override
    public void changeChannel(int channel) {
        if (isOn) {
            currentChannel = channel;
            System.out.println("Channel changed to " + currentChannel);
        } else {
            System.out.println("Cannot change channel. TV is OFF.");
        }
    }
}

// Main class to demonstrate polymorphism
public class Main {
    public static void main(String[] args) {
        // Using interface reference to control a Television object
```

```
        Remote myRemote = new Television();

        myRemote.powerOn();
        myRemote.changeChannel(5);
        myRemote.powerOff();
        myRemote.changeChannel(10); // Should warn that TV is off
    }
}
```

```
TV is now ON.
Channel changed to 5
TV is now OFF.
Cannot change channel. TV is OFF.

=== Code Execution Successful ===
```